Lecture 24

Time Complexity

Provide categories for algorithms

with respect to its scalability

- Called algorithmic complexity: “How does an algorithm scale as the size of input data increases?”
- Knowing the complexity of algorithms allows you to answer questions such as:
  - How long will a program run on an input?
  - How much space will it take?
  - Is the problem even solvable?
  - What data structure (ADT) to choose.

There are two types of algorithmic complexity

- time complexity: how the runtime of an algorithm increases as the size of the input increases
- space complexity: how the memory usage of an algorithm increases as the size of the input increases

You are running code below twice with two different inputs, input1 and input2. input1 is a list with 100 integers and input2 is a list with 1000 integers. How slower the program will be completed with input2, compared to input1?

```
input1 = […] # 100 integers
input2 = […] # 1000 integers

sum = 0
for i in range(0, len(<EITHER INPUT1 OR INPUT2>)):
        sum = sum * i
        print(i)
```

A	input1 and input2 take the same time
B	input2 takes 100 times longer
C	input2 takes 10 times longer
D	it is different every time

class[Buzz]: lec24_q1

You are running code below twice with two different inputs, input1 and input2. input1 is a list with 100 integers and input2 is a list with 1000 integers. How slower the program will be completed with input2, compared to input1?

```
input1 = […] # 100 integers
input2 = […] # 1000 integers

sum = 0
for i in range(0, len(<EITHER INPUT1 OR INPUT2>)):
    if (i%3 == 0):
        sum = sum * i
    print(i)
```

A	input1 and input2 take the same time
B	input2 takes 100 times longer
C	input2 takes 10 times longer
D	it is different every time

class[Buzz]: lec24_q1

Time complexity emphasizes “iterations”

- most of operations are done very quickly
  - arithmetic operations
  - if/else
- What it comes to time complexity, the most critical factor is loops
- Thus, calculating the number of iterations helps derive the time complexity of a algorithm

You are running code below twice with two different inputs, input1 and input2. input1 is a list with 100 integers and input2 is a list with 1000 integers. How slower the program will be completed with input2, compared to input1?

```
input1 = […] # 100 integers
input2 = […] # 1000 integers

sum = 0
for i in range(0, len(<EITHER INPUT1 OR INPUT2>)):
    for j in range(0, len(<EITHER INPUT1 OR INPUT2>)):
        sum = sum * i + j
```

A	input1 and input2 take the same time
B	input2 takes 100 times longer
C	input2 takes 10 times longer
D	it is different every time

class[Buzz]: lec24_q3

You have two stack instances, generated from an identical ADT. They are named stack A and B. After using both stacks for a while, stack A now holds 100 integers and stack B holds 1000 integers. Suppose you are pushing another integer to each stack. Which stack will complete the task quicker?

A	stack A
B	stack B
C	both will be done at the same time
D	it is different every time

class[Buzz]: lec24_q4

You have two stack instances, generated from an identical ADT. They are named stack A and B. After using both stacks for a while, stack A now holds 100 integers and stack B holds 1000 integers. Suppose you are **popping an element** each stack. Which stack will complete the task quicker?

A	stack A
B	stack B
C	both will be done at the same time
D	it is different every time

class[Buzz]: lec24_q5

You have two **linked list** instances, generated from an identical ADT. They are named linked list A and B. After using both for a while, A now holds 100 integers and B holds 1000 integers. Suppose you are **adding** another integer to **the back**. Which will complete the task quicker?

A	linked list A
B	linked list B
C	both will be done at the same time
D	it is different every time

class[Buzz]: lec24_q6

You have two **linked list** instances, generated from an identical ADT. They are named linked list A and B. After using both for a while, A now holds 100 integers and B holds 1000 integers. Suppose you are **removing** another integer from **the back**. Which will complete the task quicker?

A	linked list A
B	linked list B
C	both will be done at the same time
D	it is different every time

class[Buzz]: lec24_q7

You have two linked list instances, generated from an identical ADT. **However this time, the ADT offers “rear” reference variable.** They are named linked list A and B. After using both for a while, A now holds 100 integers and B holds 1000 integers. Suppose you are removing another integer from the back. Which will complete the task quicker?

A	linked list A
B	linked list B
C	both will be done at the same time
D	it is different every time

class[Buzz]: lec24_q8

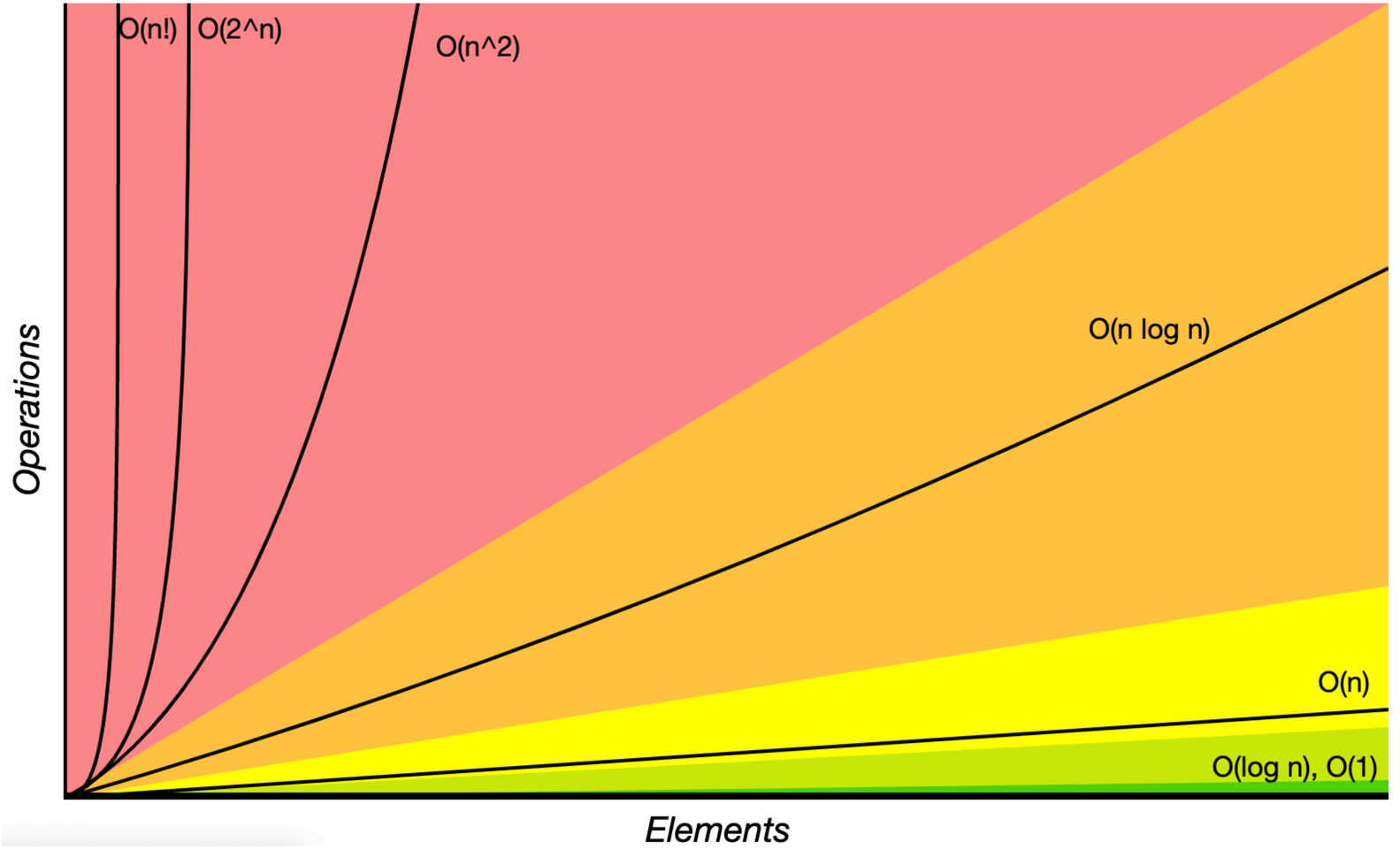
Complexity is represented as order of growth

And there are a variety of categories

- Given input size n,

$\theta(1)$	constant; the input size doesn't matter
$\theta(logn)$	logarithmic
$\theta(\sqrt{n})$	square-root
$\theta(n)$	linear; the runtime (or space) increases proportional to the input size
$\theta(n^2)$	quadratic
$\theta(c^n)$	exponential; c is a constant
$\theta(n!)$	factorial

time-complexity Graph



https://www.bigocheatsheet.com/

Some properties when calculating complexity

- Constants: Constant terms do not affect the order of growth of a process

$$\Theta(n) \qquad \Theta(500 \cdot n) \qquad \Theta(\frac{1}{500} \cdot n)$$

- Logarithms: The base of a logarithm does not affect the order of growth of a process

$$\Theta(\log_2 n) \qquad \Theta(\log_{10} n) \qquad \Theta(\ln n)$$

- Lower-order terms: The fastest-growing part of the computation dominates the total

$$\Theta(n^2) \qquad \Theta(n^2 + n) \qquad \Theta(n^2 + 500 \cdot n + \log_2 n + 1000)$$

There are three kinds of time complexities

Best/Average/Worst

- Best case: the most efficient scenario => omega notation $\Omega(\ldots)$
- Worst case: the least efficient scenario => theta notation $\theta(\ldots)$
- Average case: typical cases
- We often uses worst case in practice to conservatively estimate the time efficiency of algorithms

Big-O notation is more loose than theta

- Big-O notation, O(…), technically refers to any upper bound greater than theta
  - ex) if an algorithm has $\theta(n)$ complexity we can say that algorithm has $O(n), O(n^2), O(2^n)$, etc. because they all grow comparably or more quickly than linear complexity.
- However in practice people use O(…) and $\theta(\ldots)$ interchangeably
  - Big-O practically only refers to the tightest upper bound (=theta)

What is the time complexity of the following code?

```
sum = 0
for i in range(0, n):
    sum = sum * i

for j in range(0, n):
    sum = sum + j
```

A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)

class[Buzz]: lec24_q9

What is the time complexity of the following code?

```
sum = 0
j = n
while j>0:
    print(j)
    j = j//2
```

A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)

class[Buzz]: lec24_q10

What is the time complexity of the following code?

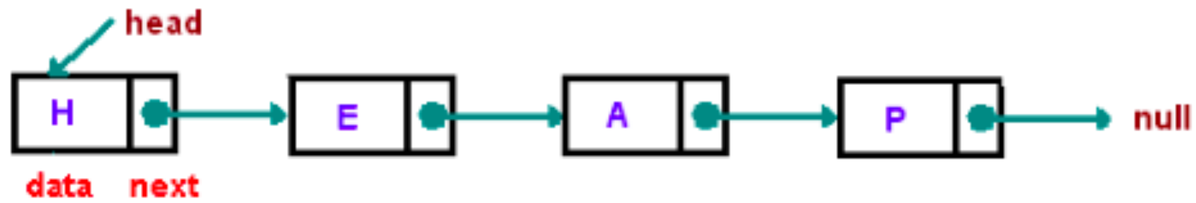
```
sum = 0

for i in range(0, n):
    j = 0
    while j <=i:
        sum = sum * i + j
        j = j + 1
```

A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)

class[Buzz]: lec24_q11

What is the time complexity of adding an element on the back of a linked list?

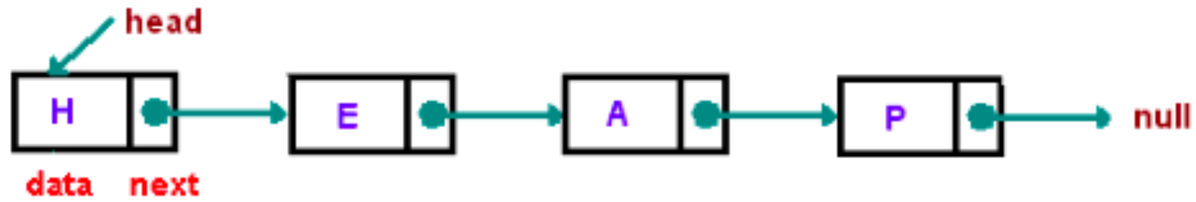
A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)



class[Buzz]: lec24_q12

What is the time complexity of **removing** an element from the back of a linked list?

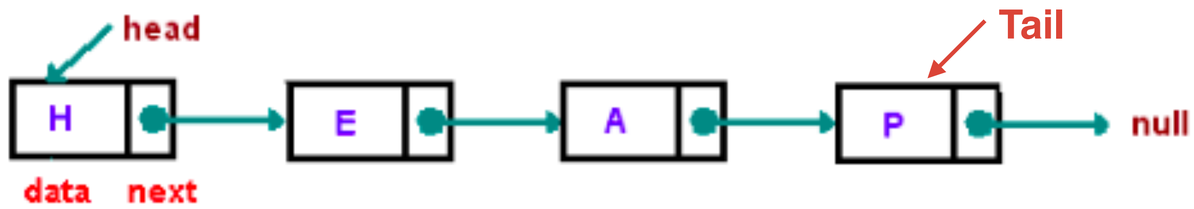
A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)



class[Buzz]: lec24_q13

What is the time complexity of **removing** an element from the back of a linked list, if this linked list offers **rear** variable?

A	O(1)
B	O(n)
C	$O(n^2)$
D	O(logn)
E	O(nlogn)



class[Buzz]: lec24_q14

What is the best- and worst-case time complexity for doing contains() (also called find( )) in a BST?

A	O(1), O(1)
B	O(1), O(log n)
C	O(log n), O(n)
D	O(n log n), O(n log n)
E	None of the above

class[Buzz]: lec24_q15

What is the best- and worst-case time complexity for doing add() in a BST?

A	O(1), O(1)
B	O(1), O(log n)
C	O(log n), O(n)
D	O(n log n), O(n log n)
E	None of the above

class[Buzz]: lec24_q16

Worst-cases

	Average cases			Worst cases		
	Add	**Remove**	**Search**	**Add**	**Remove**	**Search**
Stack				O(1)	O(1)	O(n)
Queue				O(1)	O(1)	O(n)
Liked List				the front: O(1)	the front: O(1)	O(n)
Binary Search Tree				O(n)	O(n)	O(n)
Heap				?	?	?

Worst cases & Average cases

	Average cases			Worst cases		
	Add	**Remove**	**Search**	**Add**	**Remove**	**Search**
Stack	O(1)	O(1)	O(n)	O(1)	O(1)	O(n)
Queue	O(1)	O(1)	O(n)	O(1)	O(1)	O(n)
Liked List	the front: O(1)	the front: O(1)	O(n)	the front: O(1)	the front: O(1)	O(n)
Binary Search Tree	O(logn)	O(logn)	O(logn)	O(n)	O(n)	O(n)
Heap	O(logn)	O(logn)	O(n)	O(logn)	O(logn)	O(n)